

PATRICK LIU

liu.patrick@wustl.edu | +1 (314) 760-8355 | St. Louis, MO | [linkedin.com/in/patrick-liu-2914712ba](https://www.linkedin.com/in/patrick-liu-2914712ba)

EDUCATION

Washington University in St. Louis

B.S. in Computer Science; Second Major in Mathematics

St. Louis, MO

Aug 2023 – May 2027

- GPA: 3.97 / 4.00
- Relevant Coursework: Object-Oriented Software Development, Artificial Intelligence, Machine Learning and Pattern Classification, Data Structure and Algorithms, Data Science, Computer Engineering, Web Development, Parallel and Concurrent Programming, Number Theory and Cryptography

INTERNSHIP EXPERIENCE

Virtuos

Prague, Czech Republic

Software Engineering Intern

May 2025 – Jul 2025

- Delivered 2 playable UE5 mini-game prototypes by implementing core gameplay logic and systems (actors/components, event-driven interactions) using **Unreal Engine 5** Blueprints and C++ scripting.
- Reduced ramp-up time on a new engine by self-training on UE5/Blueprint workflows in the first ~2 weeks, then rapidly iterating toward demo-ready builds under a short internship timeline.
- Improved gameplay stability and responsiveness by performing root-cause debugging (breakpoints/logs), fixing logic defects, and validating changes through frequent playtests in Unreal Editor.
- Increased development throughput and team alignment by working in Agile/Scrum (daily standups, sprint planning, demos) and incorporating peer feedback into iteration cycles.
- Maintained version-controlled progress with frequent **Git** updates to an internal repository, enabling traceable iteration and easier collaboration.

Handshake AI Fellowship

San Francisco, CA

AI Trainer & Independent Research Contributor

Aug 2025 – Sep 2025

- Improved training signal quality for **LLM** by producing and evaluating high-quality prompts, responses, and reasoning traces aligned to factuality and instruction-following goals.
- Improved evaluation consistency by applying structured verification (task completion checks, pass/fail criteria) and strictly adhering to verification heuristics and providing evidence-based rationales for edge-case pass/fail judgments.
- Strengthened model reliability efforts through iterative error analysis: identifying failure modes (hallucinations, unclear instructions, policy non-compliance) and providing targeted, actionable feedback.
- Supported quality control and audits by adhering to platform rubrics and maintaining traceable decision records.

PROJECT EXPERIENCE

Wine Quality Classification

Aug 2025 - Dec 2025

- Improved model selection reliability via running an 80/20 train-test split with 5-fold cross-validation and hyperparameter tuning via **GridSearchCV** using **Python** and **scikit-learn**.
- Increased baseline performance and training stability by standardizing features (Z-score) and re-evaluating kNN and ANN models using **NumPy** preprocessing and **scikit-learn** pipelines.
- Strengthened performance on minority wine-quality classes by addressing class imbalance with `class_weight='balanced'` in a **RandomForestClassifier** using **scikit-learn**.
- Enabled clearer model diagnostics and decision-making by visualizing feature distributions and correlations with **Pandas** and **Matplotlib**, guiding preprocessing and tuning choices.
- Delivered a multi-model comparison (**Random Forest** vs. **kNN** vs. **ANN**) by tracking metrics across experiments in **Python**, including pre/post-standardization performance and tuned vs. untuned baselines using **scikit-learn**.

Extensible File System Simulator (C++)

Jan 2025 - May 2025

- Designed and implemented a **modular file system simulator and CLI** in C++, supporting core commands including `ls` (with metadata), `rm`, `cat` (overwrite/append), `cp` (deep copy), and `rn` (rename via copy+remove).
- Increased extensibility and reduced coupling by applying **Command and Factory** patterns to decouple command execution from file object creation, keeping commands independent of concrete file types.
- Implemented the **Visitor Pattern** to separate file system operations (rendering, size calculation) from the directory structure, improving code maintainability and enabling new features without modifying existing classes.
- Improved correctness and maintainability by validating behavior through unit tests and command-line regression checks, enforcing interface-driven design and single-responsibility components.
- Integrated access control mechanisms for protected files using the **Proxy Pattern**, preventing unauthorized deep-copy or write operations.

SKILLS

- **Languages:** Python, C++, C, Java, JavaScript, R, SQL, Assembly (x86 / AVR), HTML5, CSS3, LaTeX
- **Tools/Tech:** Git, Linux/Unix Command Line, NumPy, pandas, scikit-learn, Matplotlib, React, Node.js, Unreal Engine 5